

Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    char command[200];
    pid_t pid;
    int status;

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    // Remove newline character
    command[strcspn(command, "\n")] = '\0';

    // Create child process
    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    // Child process
    if (pid == 0)
    {
        printf("\n--- Child Process ---\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());
```

```

flush(stdout);

// Execute command using shell
execl("/bin/sh", "sh", "-c", command, (char *)NULL);

// This executes only if execl() fails
perror("execl failed");
exit(1);
}

// Parent process
else
{
    printf("\n--- Parent Process ---\n");
    printf("Parent PID    : %d\n", getpid());
    printf("Child PID      : %d\n", pid);
    printf("Parent is waiting for child...\n");

    // Wait for child process to finish
    wait(&status);

    printf("Child process has finished.\n");
    printf("Parent process continues.\n");
}

return 0;
}

```

output:

--- Parent Process ---

Parent PID : 36611

Child PID : 36617

Parent is waiting for child...

--- Child Process ---

Child PID : 36617

Parent PID : 36611

total 1376

```
-rwxr-xr-x 1 pranav staff 33464 27 Jul 14:06 addition
-rw-r--r-- 1 pranav staff 173 27 Jul 14:05 addition.c
-rwxr-xr-x 1 pranav staff 33464 28 Jul 10:33 arithmetic
-rw-r--r-- 1 pranav staff 309 28 Jul 10:32 arithmetic.c
-rwxr-xr-x 1 pranav staff 34080 22 Aug 14:48 command_exec
-rw-r--r-- 1 pranav staff 1317 22 Aug 14:47 command_exec.c
drwxr-xr-x 2 pranav staff 64 27 Jul 13:42 dir3
-rwxr-xr-x 1 pranav staff 33641 22 Aug 14:46 execworking
-rw-r--r-- 1 pranav staff 209 22 Aug 14:46 execworking.c
-rw-r--r-- 1 pranav staff 9 22 Aug 14:36 f1.txt
-rw-r--r-- 1 pranav staff 9 22 Aug 14:47 f4.txt
-rwxr-xr-x 1 pranav staff 33672 4 Aug 10:34 forkid
-rw-r--r-- 1 pranav staff 733 4 Aug 10:37 forkid.c
-rwxr-xr-x 1 pranav staff 33848 17 Aug 12:14 linuxos
-rw-r--r-- 1 pranav staff 792 4 Aug 10:27 linuxos.c
-rwxr-xr-x 1 pranav staff 33569 22 Aug 14:35 multiprocess
-rw-r--r-- 1 pranav staff 165 22 Aug 14:35 multiprocess.c
-rwxr-xr-x 1 pranav staff 33600 11 Aug 10:19 multiprocess2
-rw-r--r-- 1 pranav staff 411 11 Aug 10:19 multiprocess2.c
-rwxr-xr-x 1 pranav staff 33648 11 Aug 10:34 multiprocess3
-rw-r--r-- 1 pranav staff 544 11 Aug 10:33 multiprocess3.c
-rwxr-xr-x 1 pranav staff 33464 30 Jul 11:25 process
-rw-r--r-- 1 pranav staff 191 30 Jul 11:24 process.c
-rwxr-xr-x 1 pranav staff 33464 1 Aug 15:03 process2
-rw-r--r-- 1 pranav staff 245 1 Aug 15:13 process2.c
-rwxr-xr-x 1 pranav staff 33672 1 Aug 15:38 process3
-rw-r--r-- 1 pranav staff 764 1 Aug 15:38 process3.c
-rwxr-xr-x 1 pranav staff 33432 3 Aug 12:26 process4
-rw-r--r-- 1 pranav staff 159 3 Aug 13:31 process4.c
-rwxr-xr-x 1 pranav staff 33672 11 Aug 10:32 process5
-rw-r--r-- 1 pranav staff 694 11 Aug 10:33 process5.c
-rwxr-xr-x 1 pranav staff 33464 28 Jul 10:53 switchcase
```

-rw-r--r-- 1 pranav staff 1068 28 Jul 10:53 switchcase.c

-rwxr-xr-x 1 pranav staff 33672 4 Aug 11:27 write

-rw-r--r-- 1 pranav staff 208 4 Aug 11:27 write.c

-rwxr-xr-x 1 pranav staff 33592 11 Aug 10:36 zombie

-rw-r--r-- 1 pranav staff 409 11 Aug 10:36 zombie.c

Child process has finished.

Parent process continues.